

Module 04: Exploring Spiking Neurons with Wolfram Language

Duc Nguyen

ITAI 4374: Neuroscience as a Model for AI

Spring 2026

Part A: Solving the LIF Equation with NDSolve

In[58]:=

```
Print["Wolfram Language ready!"]  
Print[$Version]
```

Wolfram Language ready!

14.3.0 for Linux x86 (64-bit) (July 8, 2025)

Wolfram Language ready!

14.3.0 for Linux x86 (64-bit) (July 8, 2025)

In[60]:=

```

ClearAll["Global`*"]

tau = 0.020;
Vrest = -70.0;
Vthresh = -55.0;
Vreset = -75.0;
R = 1.0;

Input[t_] := If[0.05 ≤ t ≤ 0.4, 18.0, 0.0];

Print["Parameters defined."]

```

Parameters defined.

Parameters defined.

Part A Knowledge Check

1. `Input[t]` returns 0.0 at $t = 0.03$ because the current has not turned on yet. It returns 18.0 at $t = 0.2$ because that time is inside the active interval from 0.05 to 0.4 seconds. It returns 0.0 at $t = 0.5$ because the current has already turned off.
2. `:=` is used so that `Input[t]` is evaluated when a value of t is supplied. This is appropriate for a time-dependent function. Using `=` would evaluate the right-hand side immediately at assignment time instead of behaving like a function of t .

In[68]:=

```

spikeTimes = {};

{sol, spikeTimes} =
  Reap[
    NDSolveValue[
      {
        tau*V'[t] == -(V[t] - Vrest) + R*Iinput[t],
        V[0] == Vrest,
        WhenEvent[V[t] ≥ Vthresh, {V[t] → Vreset, Sow[t]}]
      },
      V,
      {t, 0, 0.5}
    ]
  ];

spikeTimes = If[Length[spikeTimes] > 0, spikeTimes[[1]], {}];
spikeCount = Length[spikeTimes];

Print["Neuron fired ", spikeCount, " spikes"];
Print["Spike times: ", spikeTimes];

```

Neuron fired 8 spikes

Spike times:

```
{0.0858352, 0.126573, 0.16731, 0.208048, 0.248786, 0.289523, 0.330261, 0.370999}
```

Neuron fired 8 spikes

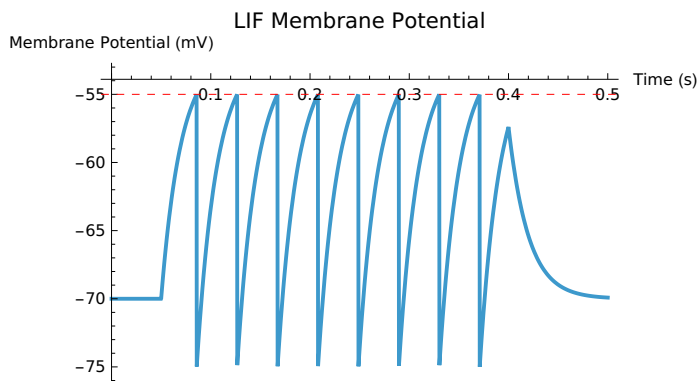
Spike times:

```
{0.0858352, 0.126573, 0.16731, 0.208048, 0.248786, 0.289523, 0.330261, 0.370999}
```

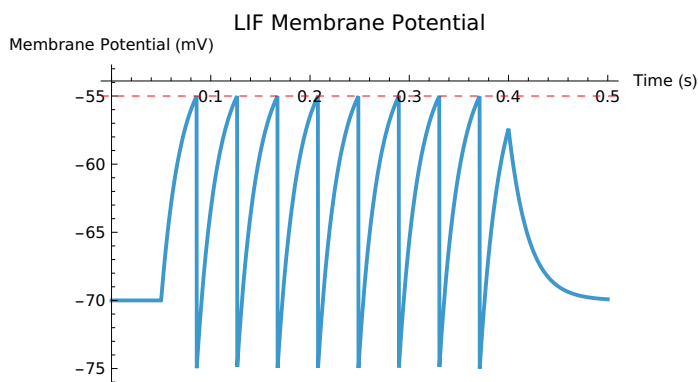
In[74]:=

```
Plot[
  sol[t],
  {t, 0, 0.5},
  PlotRange → All,
  AxesLabel → {"Time (s)", "Membrane Potential (mV)"},
  PlotLabel → "LIF Membrane Potential",
  Epilog → {Red, Dashed, InfiniteLine[{{0, Vthresh}, {1, Vthresh}}]}
]
```

Out[74]=



Out[29]=



Part A Knowledge Check

1. Between spikes, $V[t]$ follows a curve, not a straight line. This is because the LIF equation is a first-order differential equation with exponential charging behavior toward a steady-state value.
2. After the current turns off at $t > 0.4$, the membrane potential relaxes back toward V_{rest} . This happens because the input term becomes zero, so the equation reduces to passive decay back to the resting potential.

Part B: Interactive Manipulate[] Dashboard

In[75]:=

```

Manipulate[
  Module[{iFunc, solLocal, st, sc},

    iFunc[t_] := If[0.05 ≤ t ≤ 0.4, currentVal, 0.0];

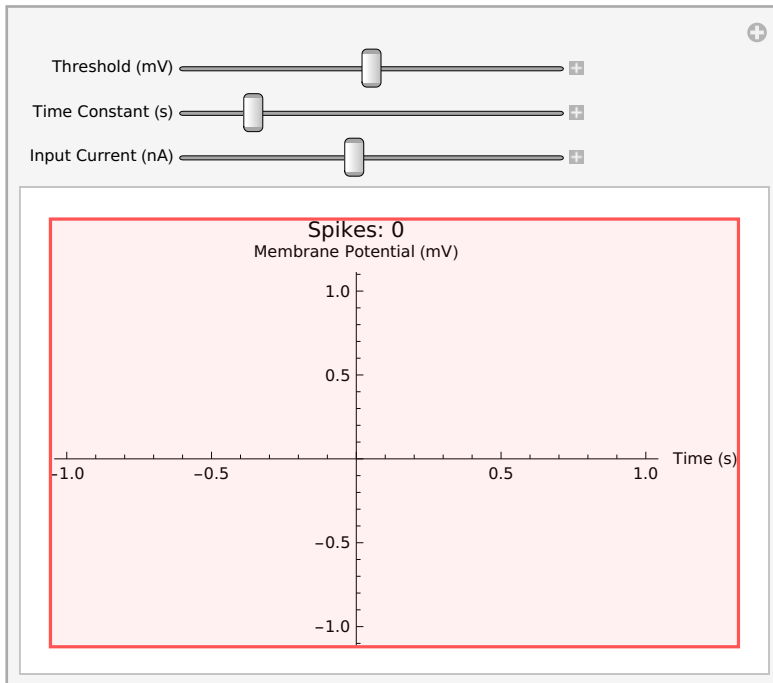
    {solLocal, st} =
      Reap[
        NDSolveValue[
          {
            tauVal*V'[t] == -(V[t] - Vrest) + R*iFunc[t],
            V[0] == Vrest,
            WhenEvent[V[t] ≥ thresh, {V[t] → Vreset, Sow[t]}]
          },
          V,
          {t, 0, 0.5}
        ]
      ];

    st = If[Length[st] > 0, st[[1]], {}];
    sc = Length[st];

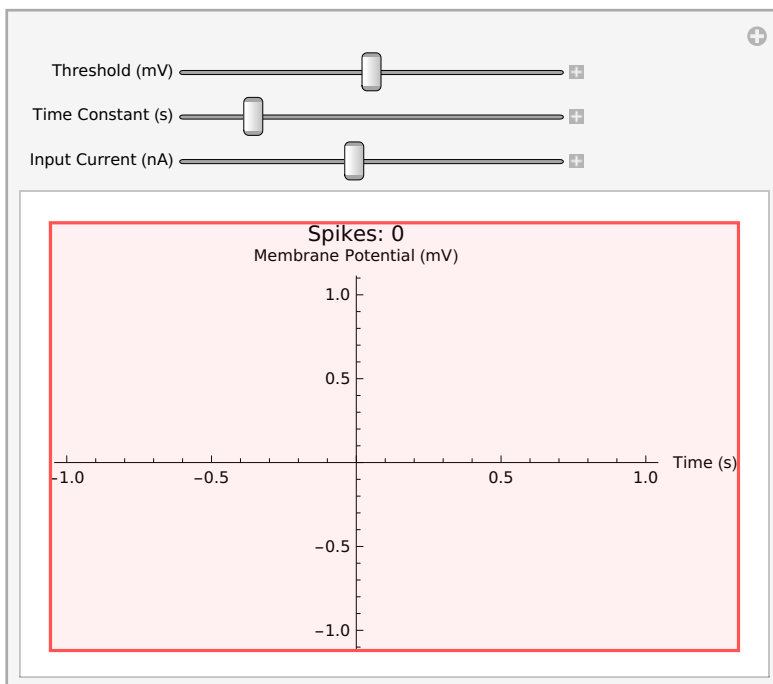
    Plot[
      solLocal[t],
      {t, 0, 0.5},
      PlotRange → All,
      AxesLabel → {"Time (s)", "Membrane Potential (mV)"},
      PlotLabel → StringForm["Spikes: ``", sc],
      Epilog → {Red, Dashed, InfiniteLine[{{0, thresh}, {1, thresh}}]}
    ]
  ],
  {{thresh, -55.0, "Threshold (mV)"}, -65, -45, 1},
  {{tauVal, 0.020, "Time Constant (s)"}, 0.005, 0.100, 0.005},
  {{currentVal, 18.0, "Input Current (nA)"}, 0, 40, 1}
]

```

Out[75]=



Out[30]=



Part B Knowledge Check

1. The rheobase is about 15 nA. Below this value the neuron does not reach threshold, and at about this level it begins to fire at least one spike.

2. With current set to 25 nA, the neuron stops firing at around -50 mV to -49 mV threshold. As the threshold becomes less negative, the neuron needs a larger depolarization to spike, so eventually the membrane potential no longer reaches threshold.

3. With threshold -55 mV and current 20 nA, $\tau = 0.010$ produces more spikes, while $\tau = 0.050$ produces a smoother and slower membrane potential curve. A smaller time constant lets the voltage change more quickly.

Reflection

The dashboard made it much easier to see that firing rate depends on several parameters at once. Reading about threshold and time constant is abstract, but moving the sliders showed immediately how a lower threshold or stronger current can increase spike rate, while a larger τ smooths and slows the response.

Part C: f-I Curve

In[76]:=

```

fiData =
  Table[
    Module[{iFunc, st, solLocal},

      iFunc[t_] := If[0.05 ≤ t ≤ 0.4, Icurr, 0.0];

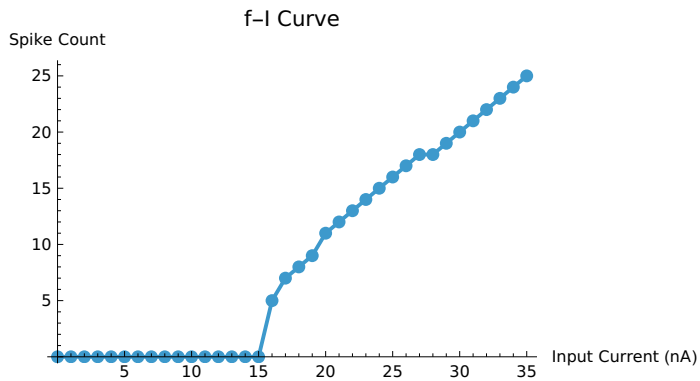
      {solLocal, st} =
        Reap[
          NDSolveValue[
            {
              tau*V'[t] == -(V[t] - Vrest) + R*iFunc[t],
              V[0] == Vrest,
              WhenEvent[V[t] ≥ Vthresh, {V[t] → Vreset, Sow[t]}]
            },
            V,
            {t, 0, 0.5}
          ]
        ];

      st = If[Length[st] > 0, st[[1]], {}];
      {Icurr, Length[st]}
    ],
    {Icurr, 0, 35, 1}
  ];

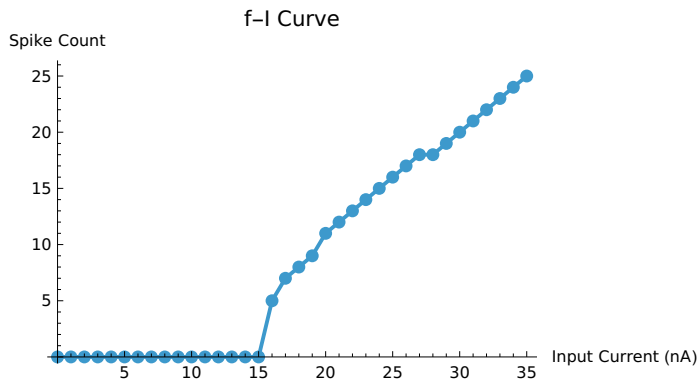
ListLinePlot[
  fiData,
  AxesLabel → {"Input Current (nA)", "Spike Count"},
  PlotLabel → "f-I Curve",
  PlotMarkers → Automatic
]

```


Out[77]=



Out[32]=



Part C Knowledge Check

1. The neuron first starts firing at about 15 nA, which is consistent with the rheobase found in the dashboard.
2. The f-I curve is not perfectly straight. It starts at zero, rises after threshold, and is roughly linear over part of the range. A perfectly straight line would mean the neuron encodes stimulus intensity with a constant proportional increase in firing rate.
3. This simple LIF model shows limited saturation compared with real neurons. Real neurons often plateau more strongly because of refractory effects, channel dynamics, and adaptation, which are not fully included here.

Part D: Encoding a Signal as Spikes

```

In[78]:= Isine[t_] := 15.0 + 12.0*Sin[2*Pi*5*t];

spikeTimesSine = {};

{solSine, spikeTimesSine} =
  Reap[
    NDSolveValue[
      {
        tau*V'[t] == -(V[t] - Vrest) + R*Isine[t],
        V[0] == Vrest,
        WhenEvent[V[t] ≥ Vthresh, {V[t] → Vreset, Sow[t]}]
      },
      V,
      {t, 0, 1.0}
    ]
  ];

spikeTimesSine = If[Length[spikeTimesSine] > 0, spikeTimesSine[[1]], {}];
spikeCountSine = Length[spikeTimesSine];

Print["Sine-input spike count: ", spikeCountSine];
Print["Spike times: ", spikeTimesSine];

```

Sine-input spike count: 19

Spike times: {0.0260992, 0.0468183, 0.0671122, 0.217907, 0.240209,
0.260038, 0.284442, 0.41797, 0.440257, 0.460087, 0.484523, 0.617971,
0.640257, 0.660087, 0.684524, 0.817971, 0.840257, 0.860087, 0.884524}

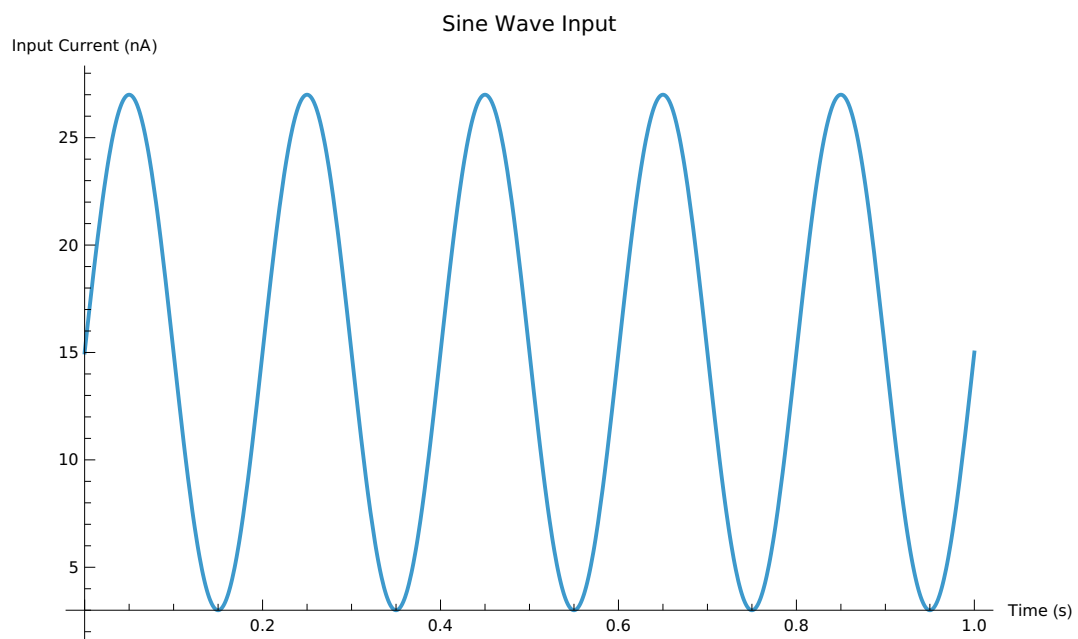
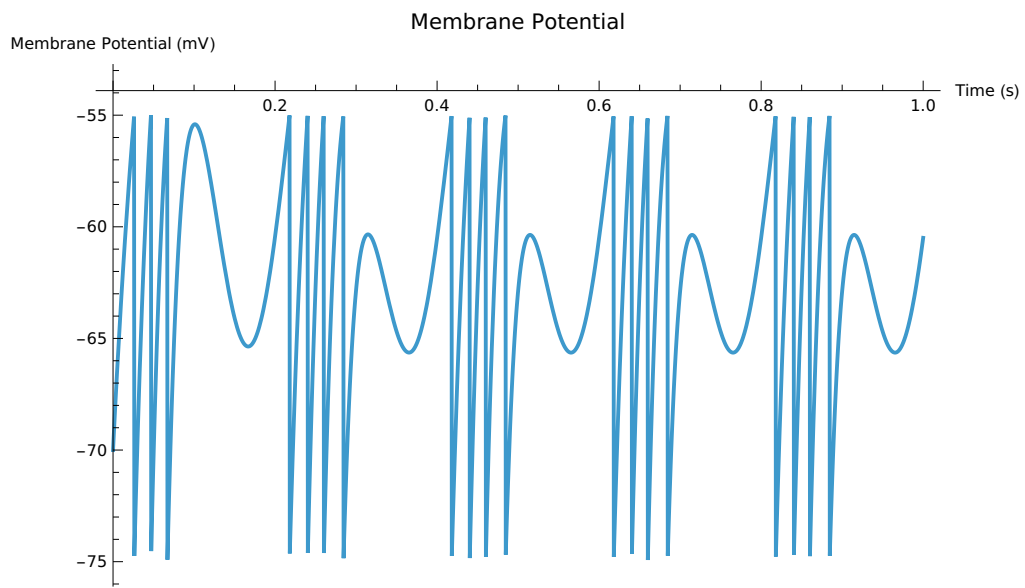
Sine-input spike count: 19

Spike times: {0.0260992, 0.0468183, 0.0671122, 0.217907, 0.240209,
0.260038, 0.284442, 0.41797, 0.440257, 0.460087, 0.484523, 0.617971,
0.640257, 0.660087, 0.684524, 0.817971, 0.840257, 0.860087, 0.884524}

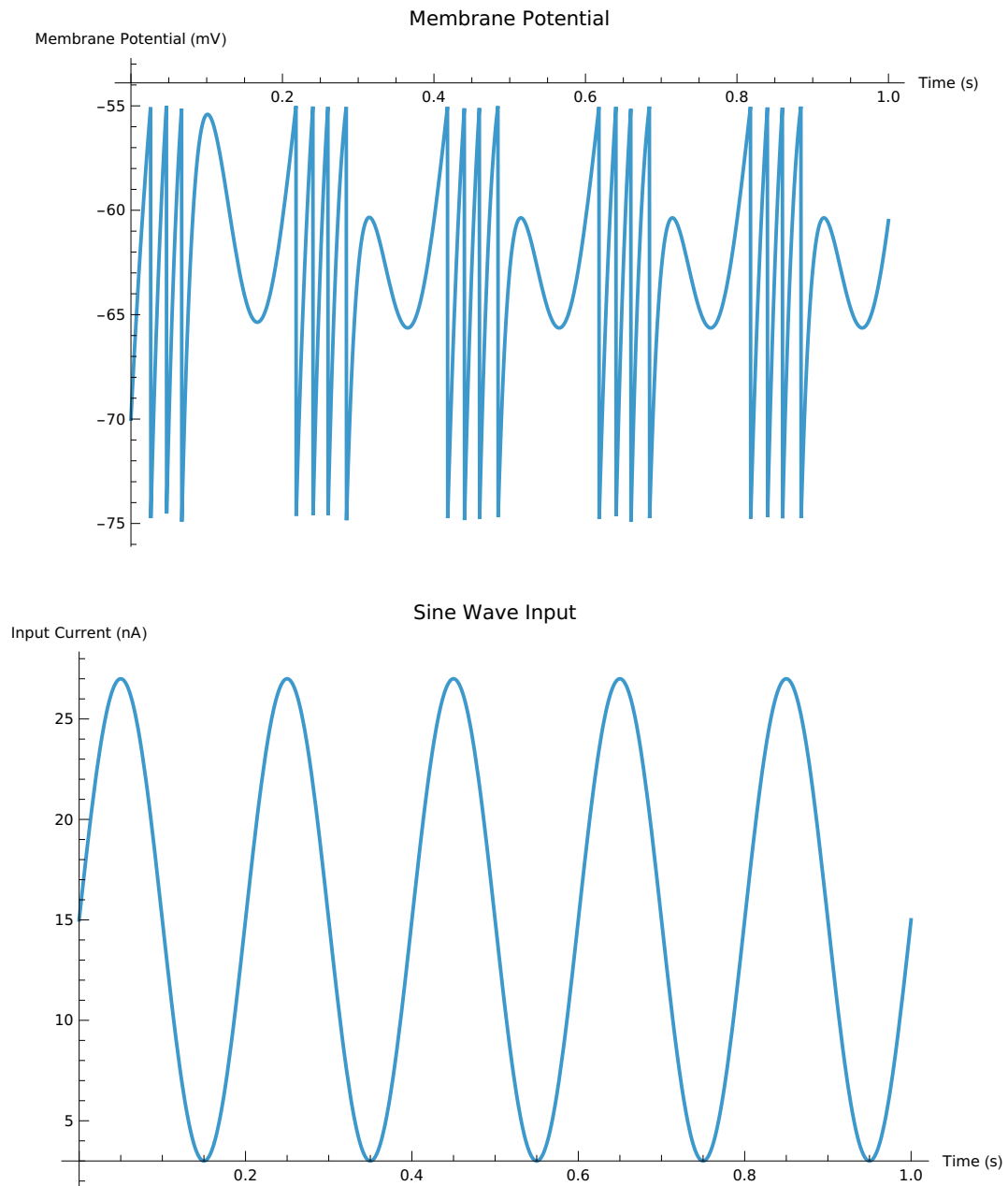
In[85]:=

```
plot1 =  
  Plot[  
    solSine[t],  
    {t, 0, 1.0},  
    PlotRange → All,  
    AxesLabel → {"Time (s)", "Membrane Potential (mV)"},  
    PlotLabel → "Membrane Potential"  
  ];  
  
plot2 =  
  Plot[  
    Isine[t],  
    {t, 0, 1.0},  
    PlotRange → All,  
    AxesLabel → {"Time (s)", "Input Current (nA)"},  
    PlotLabel → "Sine Wave Input"  
  ];  
  
GraphicsColumn[{plot1, plot2}]
```

Out[87]=



Out[42]=



Reflection

The spike train preserves the timing of when the input is strong versus weak. More spikes occur near the peaks of the sine wave, so information about signal amplitude over time is partly preserved in firing rate. What is lost is the exact smooth shape of the original analog signal, since the output is now discrete spikes rather than a continuous waveform.

Experiment 1: Threshold Sweep

Part E: Experiments

Experiment 1: Threshold Sweep

```

In[88]:= thresholdData =
  Table[
    Module[{st, solLocal},
      {solLocal, st} =
        Reap[
          NDSolveValue[
            {
              tau*V'[t] == -(V[t] - Vrest) + R*Iinput[t],
              V[0] == Vrest,
              WhenEvent[V[t] ≥ thr, {V[t] → Vreset, Sow[t]}]
            },
            V,
            {t, 0, 0.5}
          ]
        ];
      st = If[Length[st] > 0, st[[1]], 0];
      {thr, Length[st]}
    ],
    {thr, -65, -45, 5}
  ];

```

thresholdData

```

Out[89]= {{-65, 31}, {-60, 16}, {-55, 8}, {-50, 0}, {-45, 0}}

```

```

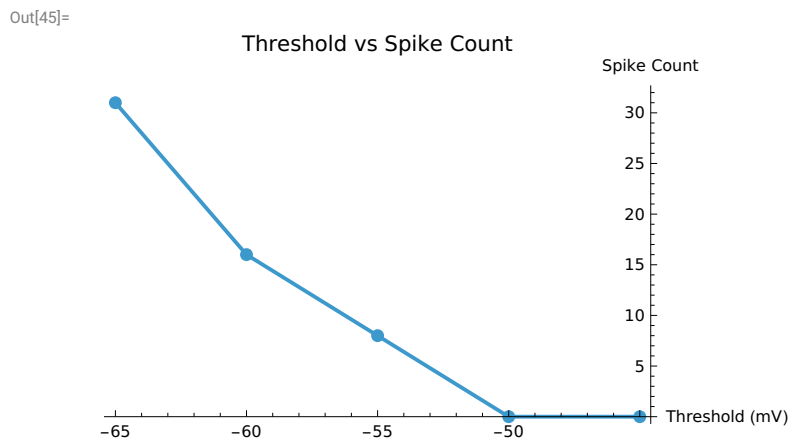
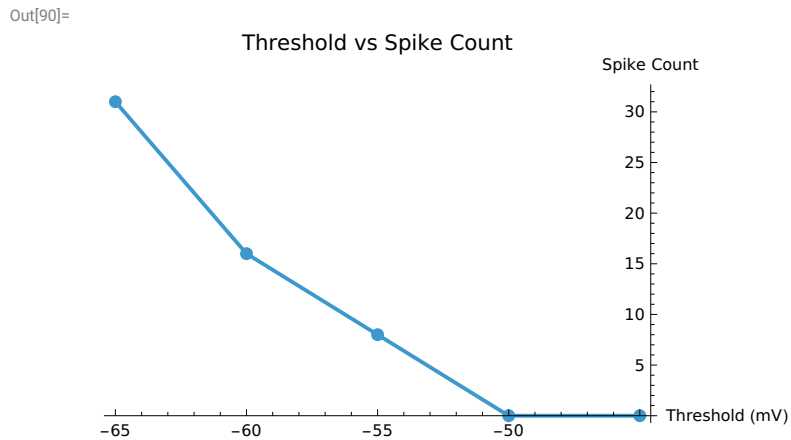
Out[44]= {{-65, 31}, {-60, 16}, {-55, 8}, {-50, 0}, {-45, 0}}

```

```

In[90]:= ListPlot[
  thresholdData,
  Joined → True,
  PlotMarkers → Automatic,
  AxesLabel → {"Threshold (mV)", "Spike Count"},
  PlotLabel → "Threshold vs Spike Count"
]

```



Experiment 1

I changed threshold from -65 to -45 in steps of 5 mV.

I expected higher thresholds to reduce firing.

The spike count decreased as threshold became less negative.

This makes sense because the membrane potential must rise farther to reach threshold.

■ Experiment 2: Time Constant Comparison

In[91]:=

```

makeTauPlot[tauTest_] :=
Module[{st, solLocal},

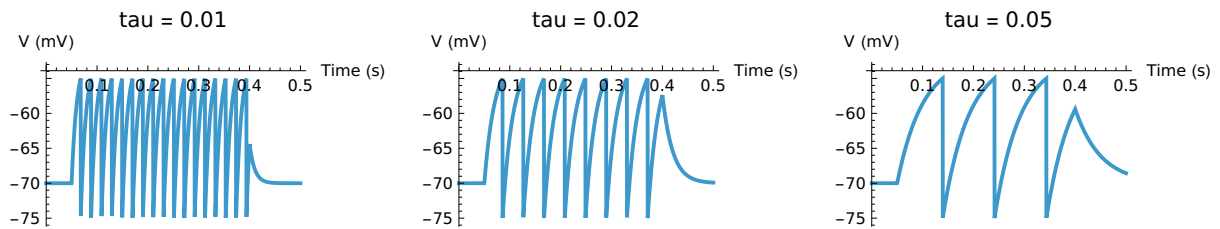
  {solLocal, st} =
    Reap[
      NDSolveValue[
        {
          tauTest*V'[t] == -(V[t] - Vrest) + R*Iinput[t],
          V[0] == Vrest,
          WhenEvent[V[t] ≥ Vthresh, {V[t] → Vreset, Sow[t]}]
        },
        V,
        {t, 0, 0.5}
      ]
    ];

  Plot[
    solLocal[t],
    {t, 0, 0.5},
    PlotRange → All,
    AxesLabel → {"Time (s)", "V (mV)"},
    PlotLabel → StringForm["tau = `", tauTest]
  ]
]

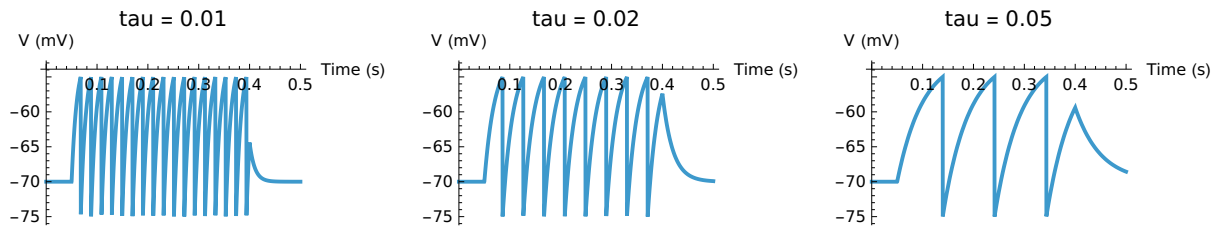
GraphicsRow[
  {
    makeTauPlot[0.010],
    makeTauPlot[0.020],
    makeTauPlot[0.050]
  }
]

```


Out[92]=



Out[47]=



Experiment 2

I changed τ to 0.010, 0.020, and 0.050.

I expected smaller τ to produce faster voltage changes and more spikes.

The $\tau = 0.010$ neuron charged fastest and fired more often, while $\tau = 0.050$ was smoother and slower.

This matches the LIF equation because a smaller membrane time constant lets voltage respond more quickly to input.

■ Experiment 4: Refractory Period

```

In[93]:= lastSpike = -1.0;
refractory = 0.003;

spikeTimesRef = {};

{solRef, spikeTimesRef} =
  Reap[
    NDSolveValue[
      {
        tau*V'[t] == -(V[t] - Vrest) + R*Iinput[t],
        V[0] == Vrest,
        WhenEvent[
          V[t] ≥ Vthresh && t - lastSpike > refractory,
          {V[t] → Vreset, lastSpike = t, Sow[t]}
        ]
      },
      V,
      {t, 0, 0.5}
    ]
  ];

spikeTimesRef = If[Length[spikeTimesRef] > 0, spikeTimesRef[[1]], {}];
Length[spikeTimesRef]

```

Out[98]=

8

Out[53]=

8

Experiment 4

I added a refractory period of 0.003 s.

I expected the maximum firing rate to decrease.

The neuron could not fire spikes as closely together, so the spike count dropped at higher input levels.

This makes sense because a refractory period prevents immediate re-firing after a spike.

■ Experiment 5: Two Different f-I Curves

In[99]:=

```

makeFiData[tauTest_] :=
  Table[
    Module[{iFunc, st, solLocal},

      iFunc[t_] := If[0.05 ≤ t ≤ 0.4, Icurr, 0.0];

      {solLocal, st} =
        Reap[
          NDSolveValue[
            {
              tauTest*V'[t] == -(V[t] - Vrest) + R*iFunc[t],
              V[0] == Vrest,
              WhenEvent[V[t] ≥ Vthresh, {V[t] → Vreset, Sow[t]}]
            },
            V,
            {t, 0, 0.5}
          ]
        ];

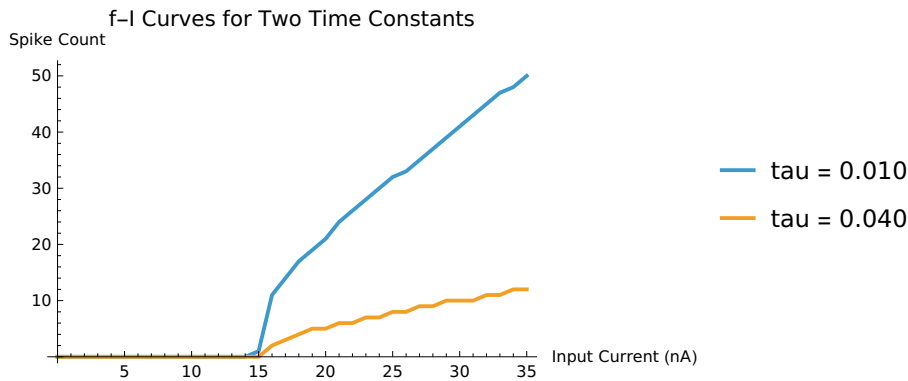
      st = If[Length[st] > 0, st[[1]], {}];
      {Icurr, Length[st]}
    ],
    {Icurr, 0, 35, 1}
  ];

fiFast = makeFiData[0.010];
fiSlow = makeFiData[0.040];

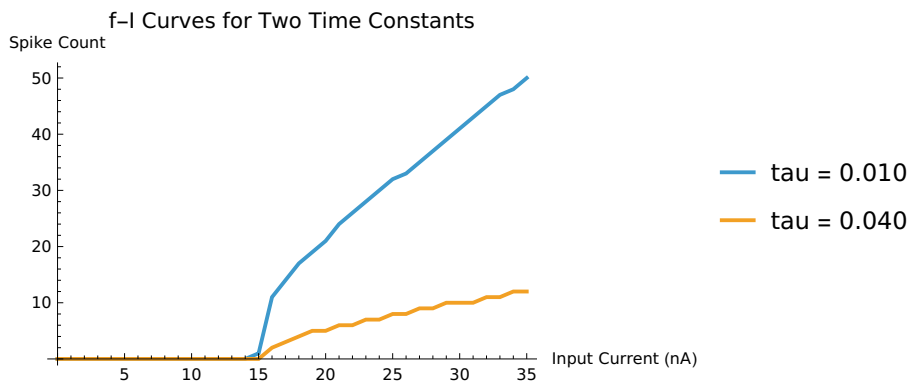
ListLinePlot[
  {fiFast, fiSlow},
  PlotLegends → {"tau = 0.010", "tau = 0.040"},
  AxesLabel → {"Input Current (nA)", "Spike Count"},
  PlotLabel → "f-I Curves for Two Time Constants"
]

```

Out[102]=



Out[57]=



Experiment 5

I compared $\tau = 0.010$ and $\tau = 0.040$.

I expected the smaller τ neuron to be more responsive.

The smaller τ curve rose earlier and more steeply.

This shows that time constant affects how quickly the neuron converts input current into spikes.

Final Reflection

1. As input current increases from below threshold to above threshold, the neuron goes from no spiking to occasional spiking and then to repeated firing. It begins firing around the rheobase, about 15 nA in this setup. This threshold behavior shows that neurons act like decision units: small inputs may do nothing, but once input is strong enough, the neuron produces an all-or-none spike.

2. The f-I curve is not perfectly linear. It has a threshold region with no firing, then a rising region where spike count increases with input. Biologically, this means stimulus intensity is not encoded

equally at all levels. Weak inputs may be ignored, while stronger inputs are translated into larger firing rates.

3. Changing the membrane time constant changes how quickly the neuron responds. A short time constant produces fast responses and usually more spikes for the same input. A long time constant produces slower, smoother voltage changes. Biologically, neurons with short time constants may be better for fast temporal tracking, while neurons with long time constants may integrate signals over longer periods.

4. The LIF model captures threshold, integration, reset, and rate coding well. It misses many biological details, including dendritic structure, multiple ion channels, synaptic neurotransmitter dynamics, refractory channel behavior in a realistic form, and spike-frequency adaptation. Real neurons are much richer than this simplified model.

5. Manipulate[] changed my understanding by making the model interactive rather than static. The most surprising discovery was how strongly a small change in threshold or time constant can change spike output. Seeing that in real time made the relation between parameters and firing behavior much more intuitive.